

ORACLE PERFORMANCE DEEP DIVE

# The Invisible Index Paradox

Why Function-Based Indexes fail without visibility.

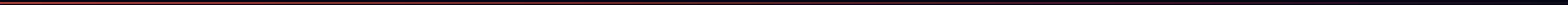
---



# The Hook



Why does the Optimizer ignore valid indexes?





# | Impact of Missing Statistics

99%

PERFORMANCE DEGRADATION

## The Cost of Blindness

When the Optimizer lacks statistics for a function-based index, it defaults to a Full Table Scan (FTS). In a 20-million-row table, this transforms a sub-second lookup into a multi-minute resource hog.



# The "Perfect" Index Failure

## Logical Validity vs. Optimizer Readiness

Building an FBI on UPPER(email) is only half the battle. If the Optimizer doesn't "see" the data distribution, it assumes a high cost and avoids the index entirely.

### Scenario:

Index Created: 11:00 PM

Full Scan Starts: 11:05 PM

Reason: Stale/Missing Stats





# Under the Hood: Virtual Columns

## Hidden Virtual Columns

When you create an FBI, Oracle silently generates a system column (e.g., SYS\_NC00010\$) to store the expression's results. The index points here, not the original column.

## The Statistics Gap

Standard GATHER\_TABLE\_STATS might miss these hidden columns unless explicitly instructed. Without histograms on these virtual values, selectivity is a mere guess.



# Pitfall 1: The Stats Blind Spot



## Missing Density

The Optimizer assumes linear distribution for unknown columns, leading to massive cardinality miscalculations.



## Stale Data

Old stats on the table don't automatically reflect the new virtual column added during FBI creation.



## The Solution

Use `method_opt => 'FOR ALL COLUMNS SIZE AUTO'` to light up the virtual column for the Optimizer.

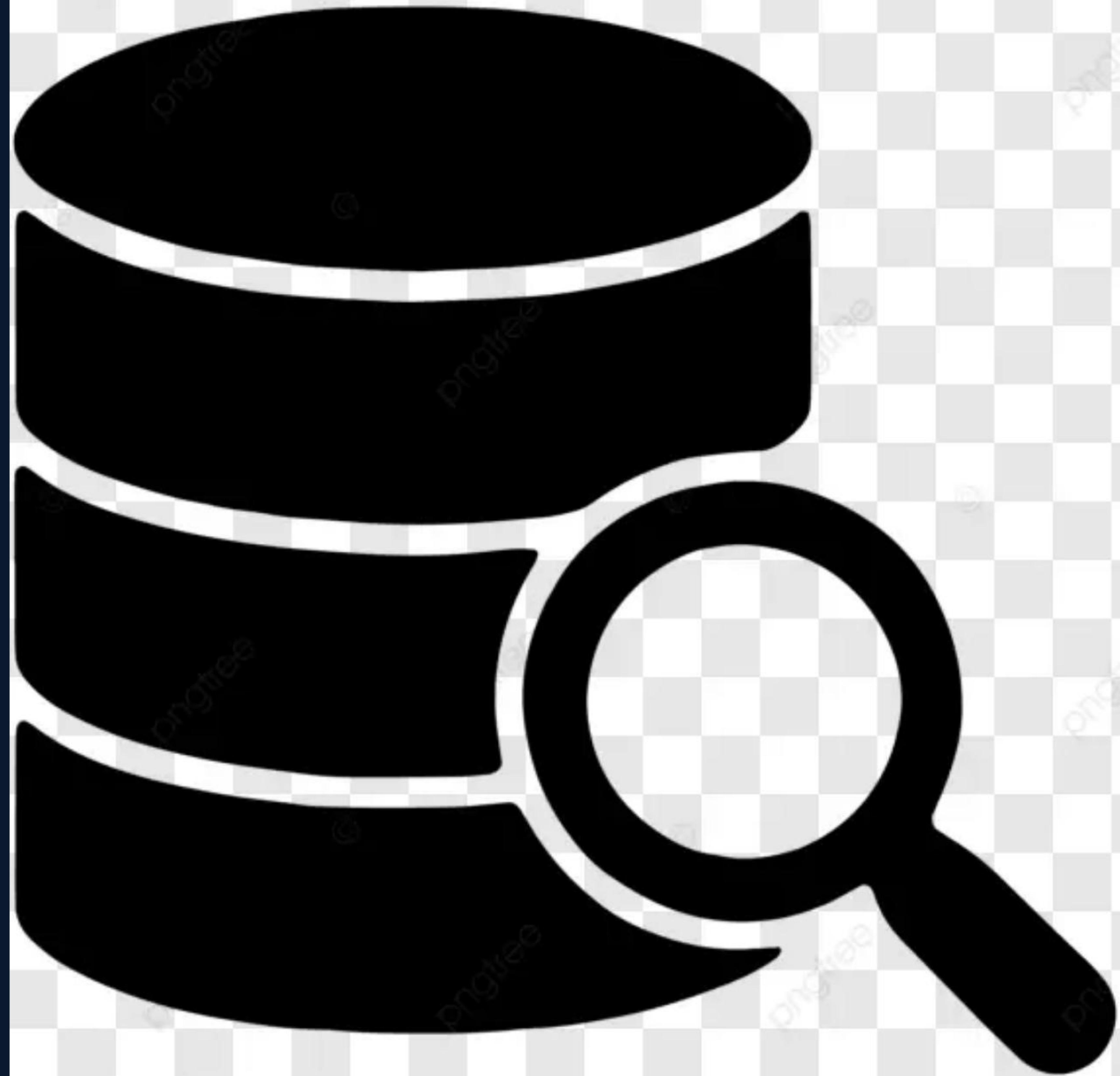


# Pitfall 2: The Null Trap

## B-Tree Index Limitations

In Oracle, a B-Tree index does not store entries where all indexed columns are NULL. If your function returns NULL, that row is effectively invisible to the index.

If your query doesn't explicitly exclude NULLs (e.g., via WHERE expression `IS NOT NULL`), the Optimizer must perform an FTS to ensure no data is missed.





# Demo Walkthrough: Path Efficiency

Full Table Scan (No Stats)

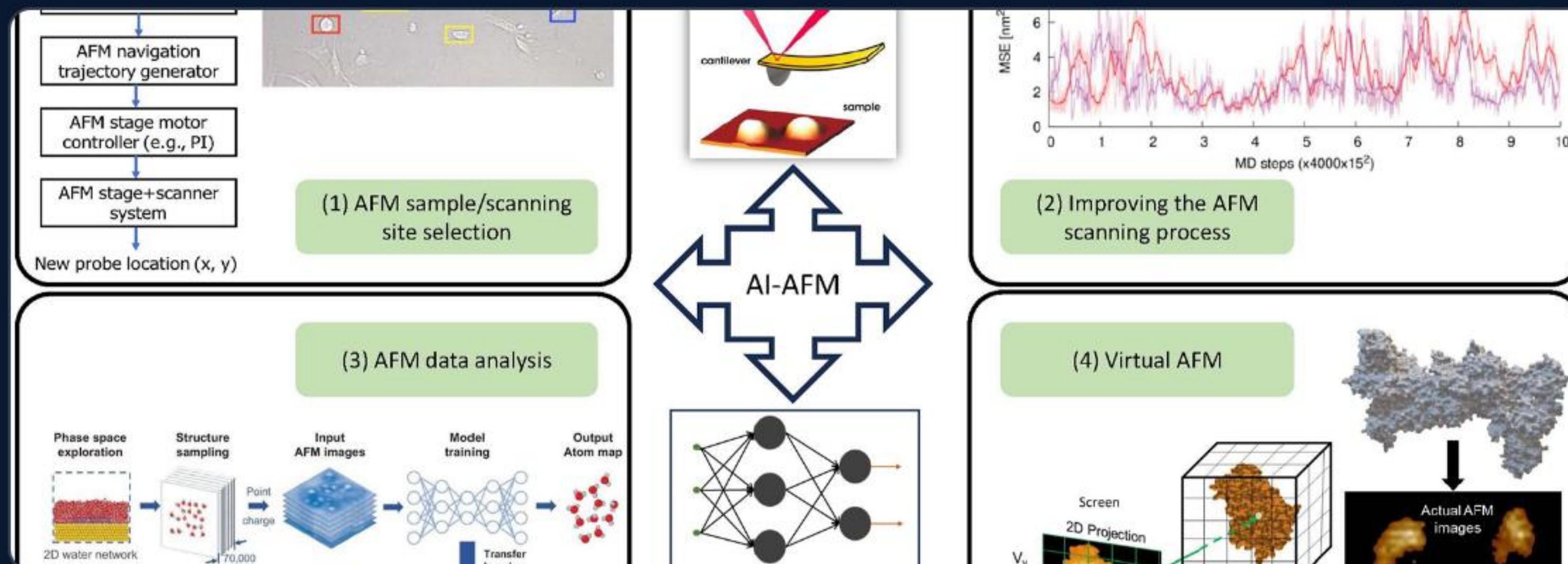
12,450 ms

FBI (Stats Missing)

9,800 ms

FBI + Optimized Stats

142 ms



*Note: Logical Reads dropped from 45,000 to 12. The execution plan shifted from TABLE ACCESS FULL to INDEX RANGE SCAN after statistics were synchronized.*

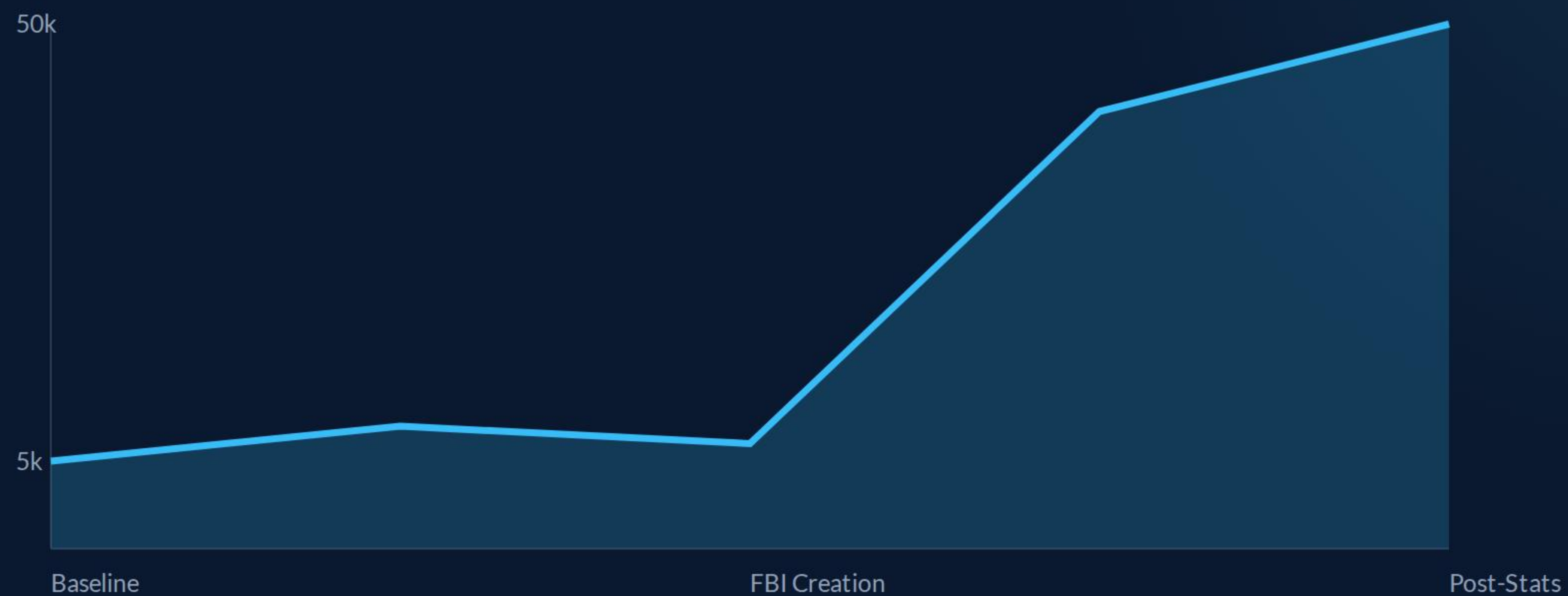


# Key Insights for Oracle DBAs

- ✓ **Always Gather Stats:** Never deploy an FBI without an immediate statistics refresh on the underlying table.
  - 🔗 **Expression Matching:** Ensure the query expression exactly matches the index expression to trigger the FBI.
  - 🔧 **Handle Nulls:** Declare indexed columns as NOT NULL or use filter predicates to guarantee index safety.
  - 🔧 **Check Virtual Names:** Use USER\_TAB\_COLS to identify the system-generated names of your FBI components.
-



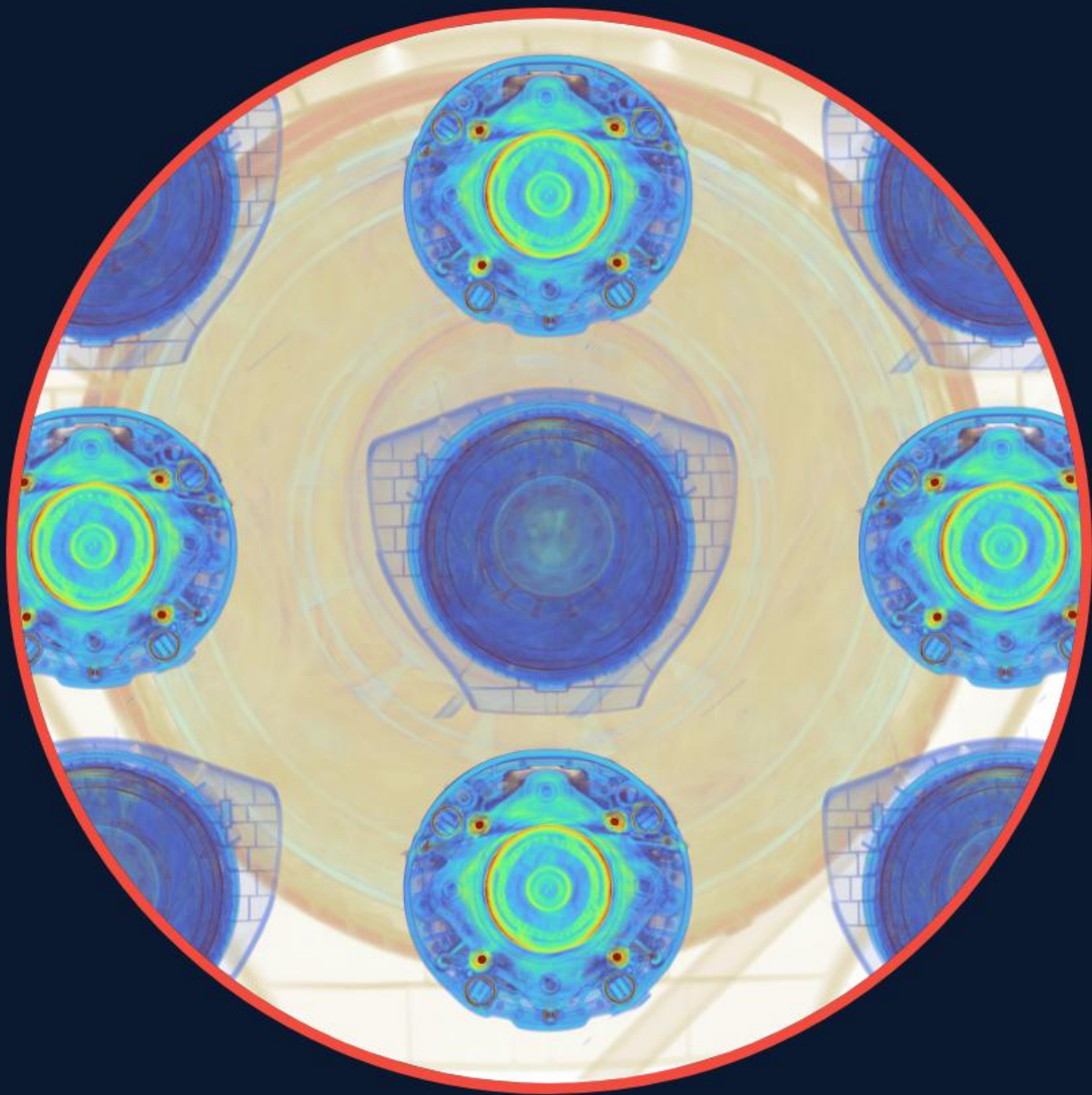
# Benefits: Resource Utilization ROI



Trend Analysis: Significant reduction in CPU and I/O Wait Time following FBI optimization. The "Post-Stats" phase shows stable sub-second performance even during peak load.



# Call to Action: The Engineer's Fix



## Implementation Checklist

Don't leave performance to chance. Follow the deterministic path to index utilization:

1. Validate index state is ENABLED.
2. Gather stats for ALL COLUMNS.
3. Verify plan with DBMS\_XPLAN.





# Summary & Closing

Function-Based Indexes are powerful, but they are not "set and forget."

**Visibility + Consistency = Performance.**

Contact your Lead DBA for deep-dive performance audits.

---



# Image Sources



[https://idnow.io/wp-content/uploads/IDnow\\_Row63-Release-Notes\\_Main\\_v2-scaled.webp](https://idnow.io/wp-content/uploads/IDnow_Row63-Release-Notes_Main_v2-scaled.webp)

Source: [idnow.io](https://idnow.io)

---



[https://png.pngtree.com/png-vector/20260206/ourlarge/pngtree-database-cylinder-icon-paired-with-a-magnifying-glass-representing-data-query-png-image\\_18708787.webp](https://png.pngtree.com/png-vector/20260206/ourlarge/pngtree-database-cylinder-icon-paired-with-a-magnifying-glass-representing-data-query-png-image_18708787.webp)

Source: [pngtree.com](https://png.pngtree.com)

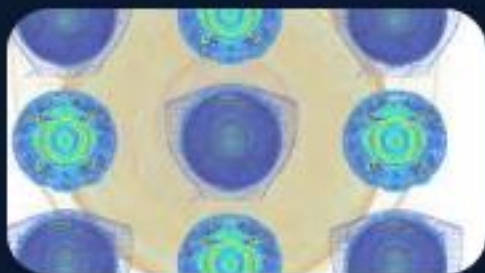
---



<https://www.frontiersin.org/files/Articles/1347648/xml-images/fphy-12-1347648-g001.webp>

Source: [www.frontiersin.org](https://www.frontiersin.org)

---



[https://cdn.prod.website-files.com/63e15418201b6e2a5cabb911/69ebef0483f86dc82fb49a2c\\_Takata.png](https://cdn.prod.website-files.com/63e15418201b6e2a5cabb911/69ebef0483f86dc82fb49a2c_Takata.png)

Source: [www.lumafield.com](https://www.lumafield.com)

---